



Department for
Digital, Culture,
Media & Sport

Aligning AWS patch management strategies with Secure by Design

Contents

Background	3
1. Patch management strategy	4
2. AWS services for patch management	5
2.1 EC2 patch management	6
2.2 Container and kubernetes patching	7
3. Serverless patch management	7
4. Automation workflows	8
5. CI/CD integration	8
6. High availability and rollback	9
7. Patching standards and best practices	9
More information	9

Background

Automated patch management helps maintain the security, stability and compliance of Amazon Web Services (AWS) environments at scale.

As government organisations grow their cloud presence across multiple AWS accounts, regions and availability zones within the UK, it becomes harder to apply patching consistently. Manual processes are difficult to sustain and often result in:

- inconsistent patching across systems
- slower remediation of vulnerabilities
- increased operational effort

These challenges increase both security and operational risks, particularly in environments with rapidly changing workloads or large, distributed infrastructure.

Secure by Design

The [Secure by Design](#) approach is a government framework which embeds cyber security into digital services from the outset. Its goal is to make security an integral, proactive part of all government digital systems, rather than an afterthought.

This document offers guidance for government organisations to align their AWS patch management practices with the [Secure by Design principles](#).

AWS-native services which support automated patch management include:

- [Systems Manager](#)
- [Amazon Inspector](#)
- [AWS Config](#)
- [AWS Security Hub](#)
- [EventBridge](#)
- [CloudWatch](#)

Used together, these services allow organisations to:

- continuously identify vulnerabilities
- automate remediation workflows
- monitor system health and patch status
- maintain visibility of compliance across the environment

Secure by Design on security.gov.uk

- [Secure by Design policy](#)
- [Implementing Secure by Design](#)
- [Guide to adopting Secure by Design](#)

1. Patch management strategy

Use immutable infrastructure

Immutable infrastructure should be the default approach to patch management where possible.

Systems should not be patched in place in production environments unless there is a clear technical reason for doing so. Instead, build updated [Amazon Machine Images](#) (AMIs) through automated pipelines. Validate them through security and functional testing, then redeploy using orchestration services such as Auto Scaling Groups or deployment pipelines.

This approach:

- reduces configuration drift
- improves reproducibility

It aligns with Secure by Design principle 7, [Minimise the attack surface](#).

For more about this, refer to the AWS Well-Architected framework page [Deploy using immutable infrastructure](#).

Build repeatable patch pipelines

Patch pipelines should be designed as repeatable build-and-release workflows, not one-off operational activities.

Pipelines should include:

- stages for patch ingestion
- dependency resolution
- OS-level application updates
- vulnerability scanning
- hardening validation
- regression testing

Only artefacts (machine image, golden image, AMI, VM template or container image) that pass all validation stages should be promoted to production. This aligns with Secure by Design principle 10, [Make changes securely](#).

Prioritise patching based on risk

Patching should be driven by a documented risk model. This should consider:

- severity of the vulnerability

- whether exploits are available
- criticality of assets
- external exposure
- data sensitivity
- business impact
- the effectiveness of compensating controls

The resulting risk rating should determine remediation priorities, timelines and deployment approaches. It should make sure internet-facing systems, critical services, and exploitable vulnerabilities are patched more quickly.

Use controlled deployment approaches

For higher-risk changes, use controlled rollout mechanisms such as canary deployments.

This means:

- updating a small number of instances first
- validating behaviour through automated health checks, monitoring and alerting
- proceeding only when the system is stable

Deployment processes should support:

- rapid detection of anomalies
- automated rollback, where necessary

Lower-risk systems can follow standard maintenance and deployment schedules.

By ensuring your remediation efforts are proportional to real-world threat exposure, your strategy will align with Secure by Design principle 3, [Adopt a risk-driven approach](#).

Define emergency patching workflows

You should define emergency patching workflows for responding to critical vulnerabilities, particularly zero-day exploits.

These workflows should enforce automated validation, logging and rollback capability. This ensures rapid response while maintaining governance visibility.

This aligns with Secure by Design principle 5, [Build in detect and respond security](#).

Enforce maintenance windows

Use automated maintenance windows to control when patching occurs. You can manage this in [AWS Systems Manager](#).

Each window should include:

- pre-change validation
- execution
- post-change verification
- a stabilisation period

Avoid poorly defined or manually executed maintenance activities in production environments, as they reduce consistency and increase risk.

This aligns with Secure by Design principle 4, [Design usable security controls](#).

2. AWS services for patch management

[AWS Systems Manager Patch Manager](#) should be the primary service for managing operating system patches across EC2 instances and hybrid environments.

It can be used to:

- define patch baselines
- schedule maintenance windows
- enforce compliance across large-scale fleets

Patch baselines must define approved and rejected updates, but should also use auto-approval rules based on factors like severity and age of patches. This helps support timely remediation.

This aligns with Secure by Design principle 3, [Adopt a risk-driven approach](#).

[AWS Systems Manager Automation](#) should be used to orchestrate end-to-end patching workflows.

These workflows typically include:

- pre-patch validation
- creating EBS snapshots
- applying patches
- coordinating reboots
- post-patch health checks

While automation should be standardised across environments, it must include rollback procedures to support recovery if a deployment fails. This aligns with Secure by Design principle 10, [Make changes securely](#).

You should enable [Amazon Inspector](#) across all supported workloads to provide continuous vulnerability scanning.

Findings should be prioritised based on:

- exploitability
- severity
- exposure context

These should feed directly into remediation pipelines.

This supports Secure by Design principle 5, [Build in detect and respond security](#).

You should use [AWS Config](#) to continuously evaluate compliance against defined patch baselines.

This helps organisations:

- detect configuration drift
- identify unmanaged resources
- keep governance aligned

This supports Principle 9: [Embed continuous assurance](#).

[AWS Security Hub](#) aggregates findings from Inspector, Config and other AWS services. This provides a unified view of security posture. This centralised model can support governance reporting and operational decision-making.

[Amazon EventBridge](#) can be used to route security findings into automated remediation workflows. It should trigger Systems Manager Automation runbooks for high-severity vulnerabilities, which means a near real-time response.

This supports Secure by Design principle 5, [Build in detect and respond security](#).

[Amazon CloudWatch](#) can provide monitoring across patch execution, system health and remediation outcomes. Alerts should be configured for:

- failure states
- degraded performance
- incomplete patch cycles

This helps teams maintain operational awareness throughout the patching process.

2.1 EC2 patch management

Onboard instances to Systems Manager

All EC2 instances should be onboarded into [AWS Systems Manager](#). The [SSM Agent](#) should be installed at launch or embedded within hardened AMIs.

Apply least-privilege principles to IAM roles so that instances can only perform the [Systems Manager](#) operations they require.

Avoid direct administrative access such as SSH wherever possible. This reduces exposure and supports Secure by Design Principle 7, [Minimise the attack surface](#).

Maintain visibility of system state

Enable AWS Systems Manager across all instances. This maintains continuous visibility of:

- installed software
- operating system versions
- running processes
- patch levels

Use this inventory for operational decision-making. Maintaining an up-to-date inventory supports Secure by Design principle 9, [Embed continuous assurance](#), by ensuring system state is always observable.

Use patch groups to manage deployments

Use patch groups to segment instances according to factors such as:

- environment (production, staging or development)
- application ownership
- compliance requirements

Each patch group should map directly to a defined patch baseline. This helps ensure deterministic behaviour during patch execution.

Enforce maintenance windows

Maintenance windows must be strictly enforced for production systems.

Activities should include automated pre-checks such as:

- CPU utilisation
- disk availability
- service dependency validation

Post-patch validation should confirm system health and application readiness before returning instances to service.

This supports Secure by Design principle 10, [Make changes securely](#).

Protect availability during patching

For highly available services, use rolling deployment strategies to ensure service continuity during patching.

Instances should be drained from load balancers before patching and only reintroduced after successful health checks.

This supports Secure by Design principle 6, [Design flexible architectures](#).

Include backup and recovery controls

Patch workflows should include backup and snapshot mechanisms.

You should create EBS snapshots before applying patches and retain them according to defined life cycle policies.

Test these backups periodically to confirm they can be restored if required.

Manage unsupported operating systems

Use automated inventory rules to identify legacy and unsupported operating systems and remove them wherever possible.

Where removal is not immediately feasible, they should be formally risk assessed and placed under heightened monitoring and governance controls.

This reduces exposure and supports Secure by Design principle 7, [Minimise the attack surface](#).

2.2 Container and kubernetes patching

Use immutable deployment practices

Container workloads should follow immutable deployment principles. Rather than modifying running containers, patching should happen through rebuilding images.

This ensures consistency across environments and eliminates configuration drift between deployments.

This aligns with Secure by Design principle 10, [Make changes securely](#) by ensuring system changes must be controlled and reproducible.

Scan container images before deployment

Enable enhanced scanning in [Amazon Elastic Container Registry](#) (ECR) to continuously evaluate container images for vulnerabilities in:

- operating system layers
- application dependencies

Integrate these findings into Continuous Integration and Continuous Delivery or Deployment (CI/CD) pipelines so that images that fail security checks are not deployed to production.

Keep EKS clusters up to date

[Amazon EKS](#) (Elastic Kubernetes Service) clusters should follow a planned upgrade life cycle that covers:

- control plane version upgrades
- node group refresh cycles
- add-on compatibility validation

Use managed node groups wherever possible so that nodes are replaced rather than patched in place.

Enforce deployment policies

Implement admission control policies using policy engines so that only compliant container images and configurations are deployed.

This enforces continuous enforcement of security controls and supports Secure By Design principle 9, [Embed continuous assurance](#).

Maintain visibility across the cluster

Integrate cluster-level observability with centralised logging and monitoring platforms. This should provide visibility of:

- workload behaviour
- deployment events
- security anomalies

3. Serverless patch management

Understand the shared responsibility model

Services such as [AWS Lambda](#) significantly reduce the amount of patching organisations need to manage. They do this by abstracting operating system and runtime maintenance responsibilities.

This helps reduce the attack surface and supports Secure by Design principle 7, [Minimise the attack surface](#).

However, application-level dependencies remain a key risk area.

Manage application dependencies

Integrate dependency scanning into CI/CD pipelines to identify vulnerabilities before deployment. These scans should evaluate:

- third-party libraries
- frameworks
- runtime dependencies

Keep Lambda runtimes up to date

You should actively manage Lambda runtime versions and upgrade them ahead of end-of-life deadlines.

Use automated alerts to identify functions using deprecated runtimes and enforce migration plans through organisational governance processes.

This supports Secure by Design principle 9, [Embed continuous assurance](#), by helping ensure systems remain supported and secure over time.

Manage secrets securely

Secrets should be stored in [AWS Secrets Manager](#) or [AWS Systems Manager Parameter Store](#).

Do not use hardcoded credentials in application code or environment variables. Using dedicated secrets management services reduces credential exposure risk and supports Secure by Design principle 10, [Make changes securely](#).

4. Automation workflows

Automation should be the primary mechanism for patch remediation at scale. [AWS Systems Manager Automation](#) can be used to standardise patching activities and make patches reusable and parameterised so they are consistent across environments.

Include validation and recovery steps

Patching workflows should include structured stages such as:

- pre-patch validation
- dependency checks
- snapshot creation
- patch execution
- service restart coordination
- post-patch validation

Where failures occur, workflows should trigger automated rollback procedures to restore previous system states.

This supports Secure By Design principle 6, [Design flexible architectures](#), by ensuring systems can recover from change.

Manage automation code

All automation documents should be version-controlled and deployed through CI/CD pipelines.

Changes should be peer-reviewed and subject to governance approval before being deployed to production. Approval gates should be used for high-risk systems or production environments.

Integrating automation pipelines with your IT service management processes can help ensure formal change control processes are enforced.

5. CI/CD integration

Where infrastructure and applications are managed through CI/CD pipelines, those pipelines should be the primary mechanism for enforcing secure patching practices.

Pipelines should include:

- operating system patching
- dependency updates
- vulnerability scanning
- configuration validation

Embedding these activities into deployment workflows helps ensure changes are applied securely and supports Secure by Design principle 10, [Make changes securely](#).

Organisations that do not use CI/CD pipelines should implement equivalent controls through alternative operational processes and tooling.

Use approved base images

[Golden AMI Pipeline](#) can help ensure consistent, hardened base images are used across all environments.

These images should be:

- rebuilt regularly
- scanned
- validated before use in production.

This reduces the attack surface, supporting Secure by Design principle 7, [Minimise the attack surface](#).

Apply security controls to container pipelines

Container pipelines should enforce security gates before workloads reach production. This helps prevent deployment of images with known vulnerabilities or outdated dependencies.

Where possible, use immutable deployment patterns so that systems are replaced with updated versions, rather than modified in place.

This approach improves predictability, reduces drift and supports Secure by Design principle 6, [Design flexible architectures](#).

6. High availability and rollback

Maintain availability during patching

Use controlled deployment strategies like rolling updates or phased rollouts to maintain high availability during patching activities.

This supports Secure by Design principle 6, [Design flexible architectures](#), by helping systems remain resilient during change.

Use blue/green deployments where appropriate

Where it is practical to maintain duplicate environments, use [blue/green deployment](#) strategies.

This allows you to:

- validate patched systems before traffic switching
- reduce operational risk
- increase confidence in the release process

Manage traffic during maintenance

Use [Elastic Load Balancing \(ELB\)](#) to route traffic away from instances undergoing patching and only reintroduce them once health checks confirm stability.

This supports Secure by Design principle 4, [Design usable security controls](#), ensuring mechanisms must support operational usability.

Integrate backup and recovery into patching

You should integrate backup and recovery mechanisms into all patch workflows, including:

- EBS snapshots

- AMI rollbacks
- database recovery procedures

Test rollback procedures

Automate rollback processes wherever possible and test them regularly. This helps ensure systems can be restored quickly and safely when patch deployments fail.

7. Patching standards and best practices

The National Cyber Security Centre (NCSC) highlights patching as a fundamental security practice for protecting systems against known vulnerabilities and reducing the likelihood of compromise.

Government organisations should establish a risk-based patch management process that covers:

- operating systems
- applications
- dependencies
- infrastructure components

The process should ensure that security updates are:

- identified
- assessed
- tested
- deployed within appropriate timescales

Effective patch management also depends on:

- asset inventory management
- vulnerability monitoring
- regular compliance reporting

Where patches cannot be applied promptly, organisations should implement appropriate mitigating controls and formally manage the associated risk until remediation is completed.

For more information, see the [NCSC guidance on patching](#).

More information

- [Secure by Design principles \(security.gov.uk\)](https://security.gov.uk/secure-by-design/principles)
- [Secure by Design activities \(security.gov.uk\)](https://security.gov.uk/secure-by-design/activities)
- [Cloud security guidance \(NCSC\)](https://www.ncsc.gov.uk/cloud-security-guidance)
- [Patching \(NCSC\)](https://www.ncsc.gov.uk/patching)
- [Managing observability \(security.gov.uk\)](https://security.gov.uk/managing-observability)